



Exercício 3.1: Manifesto de Pod completo da store-api

Aluno: Luiz Brito · Data: 18/09/2026 · Disciplina: Cloud Native Development (CP1)

Ambiente

- Cluster local **kind** v0.32.0 (1 nó control-plane), Kubernetes **v1.36.1**, contexto **kind-fiap-store**.
- Manifesto do cluster: **assets/00-kind-cluster.yaml**. Kubeconfig isolado, sem relação com outros clusters.
- Imagem **store-api:1.0** construída localmente e carregada no nó com **kind load docker-image**.
- Namespace **store-dev**.

Manifesto

Escrito à mão (sem **--dry-run** como gerador), consultando **kubectl explain** para os campos.

```
apiVersion: v1
kind: Pod
metadata:
  name: store-api
  namespace: store-dev
  labels:
    app: store-api
    tier: backend
    env: dev
spec:
  containers:
    - name: store-api
      image: store-api:1.0
      imagePullPolicy: IfNotPresent
      ports:
        - name: http
          containerPort: 8000
```

```
env:
  - name: APP_ENV
    value: dev
  - name: DB_HOST
    value: store-db.store-dev.svc.cluster.local
  - name: POD_NAME
    valueFrom:
      fieldRef:
        fieldPath: metadata.name
  - name: NODE_NAME
    valueFrom:
      fieldRef:
        fieldPath: spec.nodeName
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
startupProbe:
  httpGet:
    path: /health
    port: http
  failureThreshold: 30
  periodSeconds: 2
readinessProbe:
  httpGet:
    path: /ready
    port: http
  initialDelaySeconds: 3
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 10
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

Evidências

(a) Validação com `--dry-run=server`

O servidor valida schema, admission e defaults sem persistir o objeto.

```
$ kubectl apply --dry-run=server -f 01-pod-store-api.yaml
pod/store-api created (server dry run)
```

(b) Aplicação

```
$ kubectl apply -f 01-pod-store-api.yaml
pod/store-api created
$ kubectl wait --for=condition=Ready pod/store-api -n store-dev --timeout=90s
pod/store-api condition met
```

(c) Pod `1/1 Running`

```
$ kubectl get pod store-api -n store-dev -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE                               NOMINATED NODE
READINESS GATES
store-api     1/1     Running   0          6s    10.244.0.5    fiap-store-control-plane          <none>
<none>
```

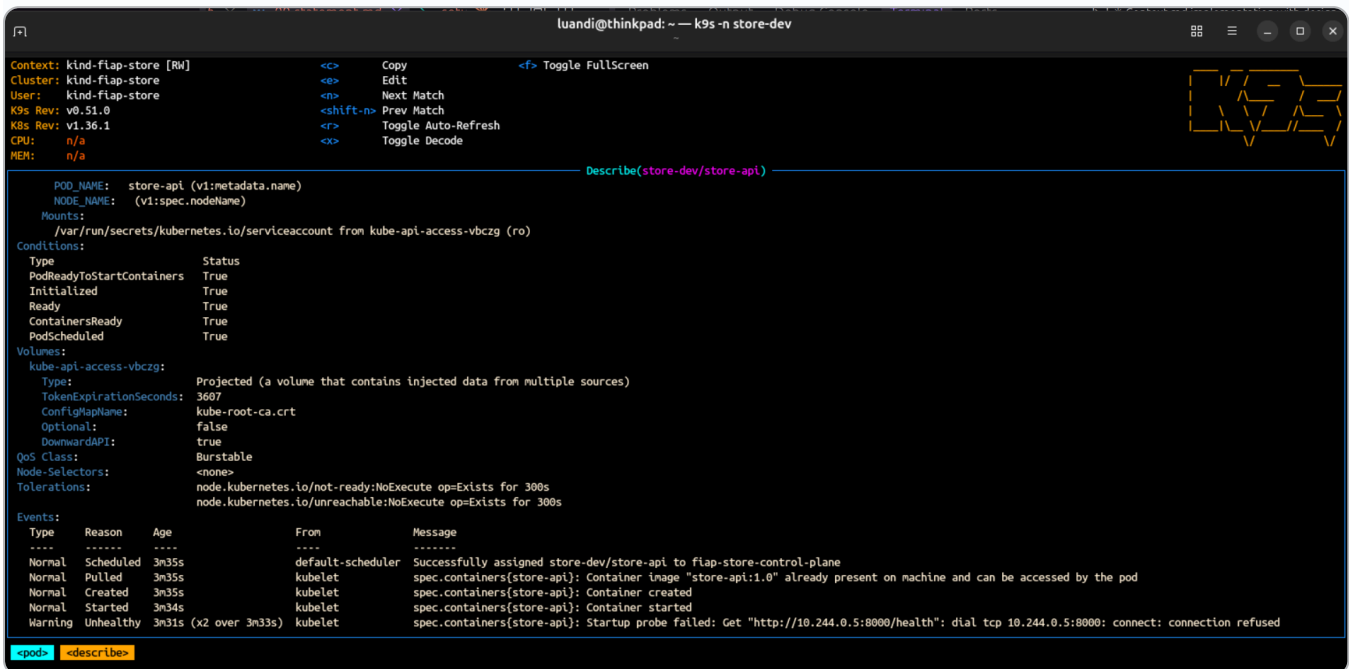


Figura 1(k9s): `store-api` com `READY 1/1`, `STATUS Running` e `0 restarts`.

(d) Classe de QoS

```
$ kubectl get pod store-api -n store-dev -o jsonpath='{.status.qosClass}{"\n"}'
```

Burstable



```
Context: kind-flap-store [RW]
Cluster: kind-flap-store
User: kind-flap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a

Describe(store-dev/store-api)

POD_NAME: store-api (v1:metadata.name)
NODE_NAME: (v1:spec.nodeName)
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vbczg (ro)
Conditions:
  Type             Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-vbczg:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations:
  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason      Age    From          Message
  ----    -
  Normal  Scheduled   3m35s  default-scheduler  Successfully assigned store-dev/store-api to flap-store-control-plane
  Normal  Pulled      3m35s  kubelet        spec.containers(store-api): Container image "store-api:1.0" already present on machine and can be accessed by the pod
  Normal  Created     3m35s  kubelet        spec.containers(store-api): Container created
  Normal  Started     3m34s  kubelet        spec.containers(store-api): Container started
  Warning  Unhealthy   3m31s  (x2 over 3m33s)  kubelet        spec.containers(store-api): Startup probe failed: Get "http://10.244.0.5:8000/health": dial tcp 10.244.0.5:8000: connect: connection refused
```

Figura 2 (k9s, describe): QoS Class: Burstable.

Justificativa. O Pod é **Burstable** porque tem **requests** e **limits** definidos, mas com valores diferentes (CPU 100m ≠ 500m e memória 128Mi ≠ 256Mi). Não é **Guaranteed**, que exige **requests = limits** em **todos** os recursos de **todos** os containers. Também não é **BestEffort**, que só existe quando nenhum container declara **requests / limits**.

(e) As três probes no describe

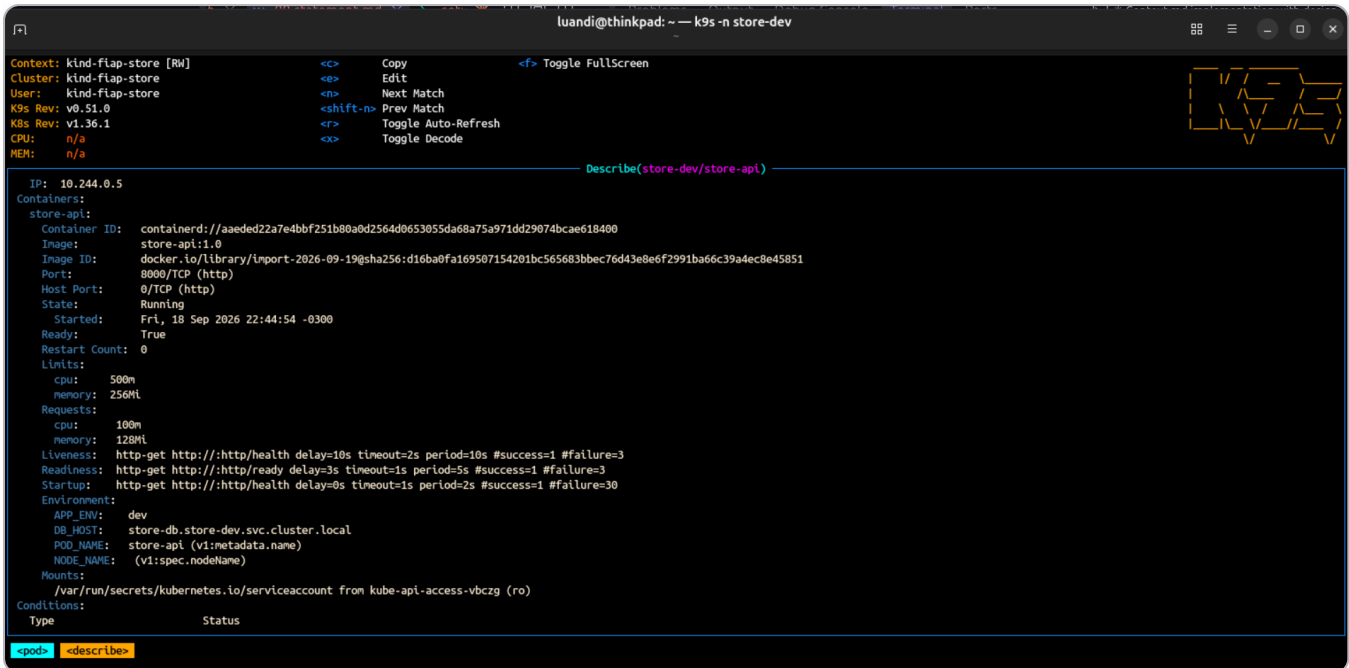
```
$ kubectl describe pod store-api -n store-dev | grep -E
'QoS|Liveness|Readiness|Startup|Limits|Requests|cpu|memory'

Limits:
  cpu:    500m
  memory: 256Mi

Requests:
  cpu:    100m
  memory: 128Mi

Liveness:  http-get http://:http/health delay=10s timeout=2s period=10s #success=1 #failure=3
Readiness:  http-get http://:http/ready delay=3s timeout=1s period=5s #success=1 #failure=3
Startup:    http-get http://:http/health delay=0s timeout=1s period=2s #success=1 #failure=30
QoS Class:  Burstable

Warning Unhealthy 7s (x2 over 9s) kubelet spec.containers(store-api): Startup probe
failed: Get "http://10.244.0.5:8000/health": dial tcp 10.244.0.5:8000: connect: connection refused
```



```
luandi@thinkpad: ~ -- k9s -n store-dev
Context: kind-flap-store [RW]
Cluster: kind-flap-store
User: kind-flap-store
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a

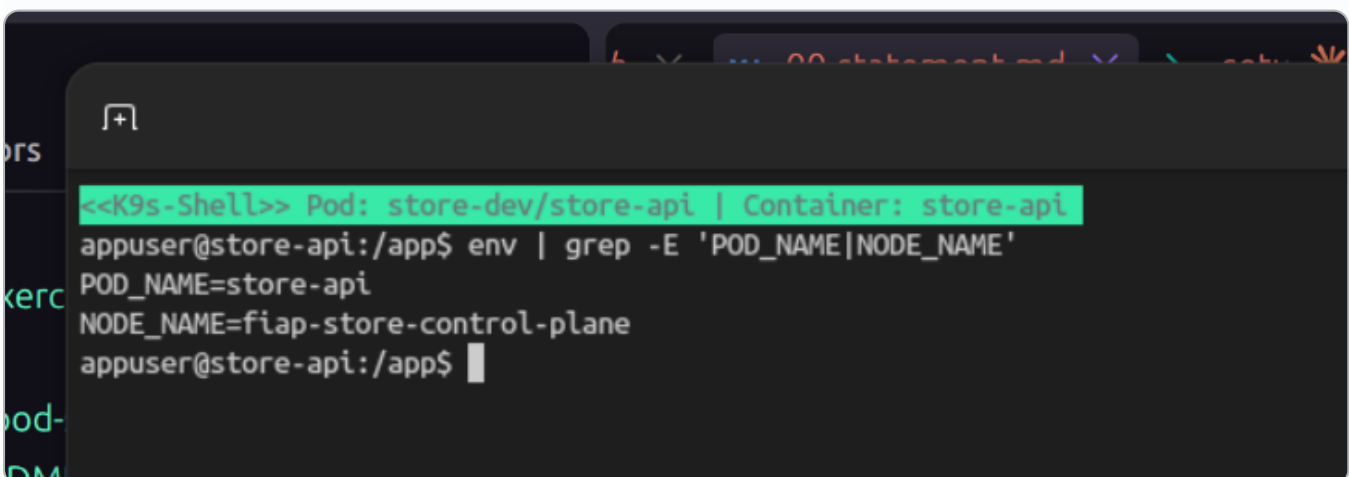
IP: 10.244.0.5
Containers:
store-api:
  Container ID: containerd://aaeded22a7e4bbf251b80a0d2564d0653055da68a75a971dd29074bcae618400
  Image: store-api:1.0
  Image ID: docker.io/library/import-2026-09-19@sha256:d16ba0fa169507154201bc565683bbec76d43e8ef2991ba66c39a4ec8e45851
  Port: 8000/TCP (http)
  Host Port: 0/TCP (http)
  State: Running
  Started: Fri, 18 Sep 2026 22:44:54 -0300
  Ready: True
  Restart Count: 0
  Limits:
    cpu: 500m
    memory: 256Mi
  Requests:
    cpu: 100m
    memory: 128Mi
  Liveness: http-get http://:http/health delay=10s timeout=2s period=10s #success=1 #failure=3
  Readiness: http-get http://:http/ready delay=3s timeout=1s period=5s #success=1 #failure=3
  Startup: http-get http://:http/health delay=0s timeout=1s period=2s #success=1 #failure=30
  Environment:
    APP_ENV: dev
    DB_HOST: store-db.store-dev.svc.cluster.local
    POD_NAME: store-api (v1:metadata.name)
    NODE_NAME: (v1:spec.nodeName)
  Mounts:
    /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-vbczg (ro)
Conditions:
  Type Status
<pod> <describe>
```

Figura 3 (k9s, `describe`): `Liveness`, `Readiness` e `Startup`, todas via `http-get` na porta `http` (referência pelo nome).

O evento `Startup probe failed` é esperado: a aplicação ainda estava subindo quando o kubelet fez as duas primeiras tentativas. A `startupProbe` segura as demais probes até o primeiro sucesso, então não houve reinício (`RESTARTS 0`).

(f) Downward API: `POD_NAME` e `NODE_NAME`

```
$ kubectl exec store-api -n store-dev -- env | grep -E 'POD_NAME|NODE_NAME'
POD_NAME=store-api
NODE_NAME=fiap-store-control-plane
```



```
<<K9s-Shell>> Pod: store-dev/store-api | Container: store-api
appuser@store-api:/app$ env | grep -E 'POD_NAME|NODE_NAME'
POD_NAME=store-api
NODE_NAME=fiap-store-control-plane
appuser@store-api:/app$
```

Figura 4 (k9s, shell no container `store-api`): as variáveis chegam com o nome real do Pod e do nó.

(g) Tempo máximo para matar o container com a liveness falhando

A liveness usa `initialDelaySeconds: 10`, `periodSeconds: 10` e `failureThreshold: 3`. Tempo máximo = $\text{initialDelaySeconds} + \text{failureThreshold} \times \text{periodSeconds} = 10 + 3 \times 10 = \mathbf{40 \text{ segundos}}$. Depois disso o kubelet mata o container e a `restartPolicy` (padrão `Always`) o reinicia.

Conclusão

Os sete itens foram cumpridos: o manifesto passou na validação do servidor, o Pod ficou `1/1 Running`, a QoS resultante é `Burstable` (com justificativa), as três probes aparecem no `describe` e as variáveis da Downward API chegaram corretas ao container.